

# Fenwick Tree (Binary Indexed Tree) Method for Range Sum Queries

## 1 Given Array

1-indexed array:

$$A = [2, 3, 6, 7, 5]$$

## 2 Step 1: Build the Fenwick Tree (BIT)

The BIT array stores sums for ranges defined by the least significant bit (LSB):

$$\text{LSB}(i) = i \& (-i)$$

**BIT construction:**

| $i$ | Range Covered | $BIT[i]$        | <i>Explanation</i>          | <i>Value</i> |
|-----|---------------|-----------------|-----------------------------|--------------|
| 1   | [1, 1]        | 2               | <i>Only</i> $A[1]$          | 2            |
| 2   | [1, 2]        | $2 + 3$         | <i>Sum of</i> $A[1] + A[2]$ | 5            |
| 3   | [3, 3]        | 6               | <i>Only</i> $A[3]$          | 6            |
| 4   | [1, 4]        | $2 + 3 + 6 + 7$ | <i>Sum of</i> $A[1]..A[4]$  | 18           |
| 5   | [5, 5]        | 5               | <i>Only</i> $A[5]$          | 5            |

$$\text{BIT} = [2, 5, 6, 18, 5]$$

## 3 Step 2: Prefix Sum Function Using BIT

To compute prefix sum up to index  $i$ :

$$\text{prefix\_sum}(i) = \sum_{j=i}^0 BIT[j], \quad j = j - \text{LSB}(j)$$

—

## 4 Query 1: Sum(2,4)

Formula using Fenwick Tree:

$$\text{sum}(L, R) = \text{prefix\_sum}(R) - \text{prefix\_sum}(L - 1)$$

**Step 1: prefix\_sum(4)**

$$\begin{aligned}
i &= 4, \quad sum = 0 \\
sum &+ = BIT[4] = 18 \\
i &= 4 - LSB(4) = 4 - 4 = 0 \quad (\text{stop}) \\
\text{prefix\_sum}(4) &= 18
\end{aligned}$$

**Step 2: prefix\_sum(1)**

$$\begin{aligned}
i &= 1, \quad sum = 0 \\
sum &+ = BIT[1] = 2 \\
i &= 1 - LSB(1) = 0 \quad (\text{stop}) \\
\text{prefix\_sum}(1) &= 2
\end{aligned}$$

**Step 3: Compute sum(2,4)**

$$\text{sum}(2, 4) = \text{prefix\_sum}(4) - \text{prefix\_sum}(1) = 18 - 2 = \boxed{16}$$

**Verification:**

$$3 + 6 + 7 = 16$$

—

**5 Query 2: Sum(2,5)****Step 1: prefix\_sum(5)**

$$\begin{aligned}
i &= 5, \quad sum = 0 \\
sum &+ = BIT[5] = 5 \\
i &= 5 - LSB(5) = 4 \\
sum &+ = BIT[4] = 18 \\
i &= 4 - LSB(4) = 0 \quad (\text{stop}) \\
\text{prefix\_sum}(5) &= 23
\end{aligned}$$

**Step 2: prefix\_sum(1)**

$$\text{prefix\_sum}(1) = 2 \quad (\text{from previous calculation})$$

**Step 3: Compute sum(2,5)**

$$\text{sum}(2, 5) = \text{prefix\_sum}(5) - \text{prefix\_sum}(1) = 23 - 2 = \boxed{21}$$

**Verification:**

$$3 + 6 + 7 + 5 = 21$$

—

**6 Notes**

- Fenwick Tree efficiently computes prefix sums in  $O(\log n)$
- Range sum = `prefix_sum(R) - prefix_sum(L-1)`
- Point updates are applied by adding value to BIT indices affected by LSB
- Space complexity:  $O(n)$